BECODE-2026 · INTERVIEW DEMO GUIDE

How the demo works, step by step

What each command does, what happens inside the tools, and what to say.

ioc-enricher → mini-siem → mini-soar → you, the analyst

What this guide is for

The interview demo runs with one command, `.\demo.ps1`. This guide explains what happens at each of its five steps, so you understand it well enough to answer any question about it. Each step has the same parts:

You run	the command the script runs (and that you could type yourself)
What happens inside	what each tool does with the data, in order
What you see	the real output or screen, with the important parts explained
Why it matters	the analyst thinking behind it: this is what interviewers evaluate
What to say	short points to make while the step runs

Contents

1. The big picture · 2. The incident used in the demo · 3. What `demo.ps1` does · Steps 1 to 5 · After the demo · Glossary · If something goes wrong

1. The big picture

Your three projects form a small version of what a real SOC (Security Operations Centre) runs: logs come in, suspicious activity is detected, its indicators are checked against threat intelligence, simple responses are automated, and a human analyst makes the decisions that matter.



Tool	Its job in one sentence	Real-world equivalent
mini-SIEM	**Sees.** Collects logs, puts them in one format, finds attacks with rules, and shows alerts in a dashboard.	Splunk, Microsoft Sentinel, Elastic, Wazuh
ioc-enricher	**Checks.** Asks threat-intelligence services whether each IP, domain, URL or file hash is known to be malicious.	The enrichment step of any SIEM or SOAR
mini-SOAR	**Acts.** Runs playbooks on alerts: blocks confirmed threats, opens a case, notifies the team, and asks a human before anything disruptive.	Splunk SOAR, Cortex XSOAR, Tines, Shuffle

Why it matters. A SOC receives far more alerts than people can check by hand. The SIEM finds them, enrichment adds context, the SOAR does the repetitive work, and analysts spend their time on judgment. Showing you understand this split is the main message of the demo.

2. The incident used in the demo

Everything runs on one fictional incident called **Paper Lantern**, at an invented company, *Lumen Freight*. Each stage leaves traces in a different log source, and each trace triggers a rule:

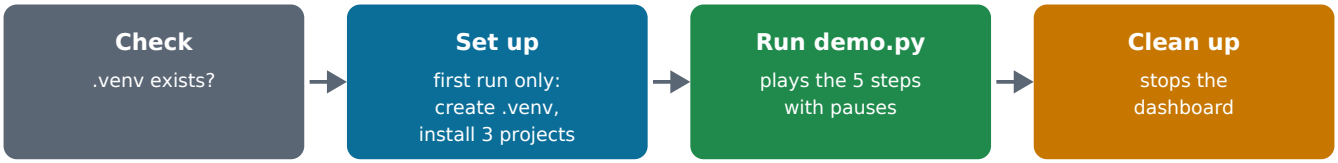
Time	What happens	Seen in	Alert
08:41	A finance employee receives a fake overdue-invoice email from a lookalike domain	Mail gateway	MS-001
08:44	They download `INV-2026-1043.pdf.js`: a script disguised as a PDF	Web proxy	MS-002
08:45	The script runs, writes a program to a shared folder, and starts it	EDR	MS-003, 004, 005
08:46	The program contacts the same server every 60 seconds (command-and-control)	Firewall	MS-006
09:30	It downloads an extra module from a server addressed by IP, and loads it	Proxy, EDR	MS-008, 007
09:52	The workstation is isolated	EDR	–

A second workstation produces normal activity that *looks* suspicious (frequent Teams connections, a JavaScript library from a CDN, a logon script). It must not raise any alert: that is how the demo shows the rules are precise, not just noisy.

3. What `demo.ps1` does

You start everything with one command from the `Becode-2026` folder:

```
$ .\demo.ps1 -Name mateo
```



1. **Checks the environment.** It looks for a `.venv` folder (a private Python installation just for this repository). On the first run, it creates it and installs ioc-enricher, mini-siem and mini-soar into it. This takes about a minute and only happens once.
2. **Hands over to demo.py.** The PowerShell file only prepares things. The demo itself is a Python script, so it also works on Mac and Linux, and GitHub tests it on Windows and Linux at every push.
3. **Plays the five steps.** For each step it prints the command as if you typed it, runs it, then shows the points to make and waits for **Enter**. Type **q** then Enter to stop at any time.
4. **Runs the dashboard for you.** In step 3 it starts the mini-SIEM web dashboard in the background (on port 5000, or the next free port), and opens your browser on the right page at each moment.
5. **Always cleans up.** At the end, or if you stop early, it shuts the dashboard down.

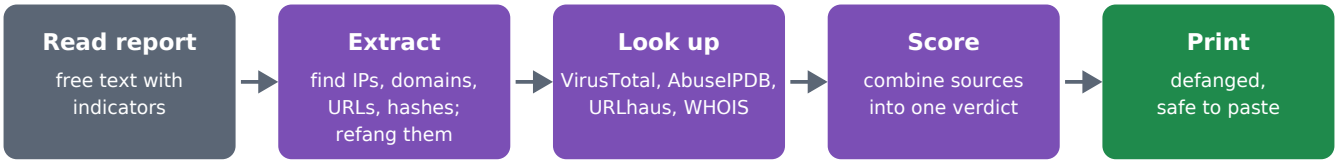
Option	Effect
`-Name mateo`	Your name, recorded when you approve or reject actions
`-NoPause`	Runs straight through: use it to rehearse
`-NoBrowser`	Doesn't open the browser
`-SkipTests`	Skips step 5
`-Setup`	Reinstalls the projects, for example after a git pull

STEP 1 OF 5 ## Threat-intel enrichment with ioc-enricher ~1–2 min

YOU RUN

```
$ ioc-enricher -x -f tests\data\paper_lantern_report.txt --offline-dataset
tests\data\paper_lantern_dataset.json
```

WHAT HAPPENS INSIDE



1. **Extraction (-x).** The tool reads an incident report written in normal sentences and pulls out every indicator. It understands *defanged* notation like `hxxp://evil[.]com` and ignores file names such as `invoice.pdf.js`.
2. **Lookups.** Each indicator goes to the sources that support its type. `--offline-dataset` makes the tool answer from a file of invented responses, so the demo needs no internet or API keys. Internal IPs are never looked up.
3. **Scoring.** Each source gives a score from 0 to 100. The verdict uses the highest score, plus a bonus when several sources agree: 70+ is MALICIOUS, 30+ SUSPICIOUS, below that CLEAN, and no data at all is UNKNOWN.

WHAT YOU SEE

```
[MALICIOUS 100] invoice-portal-secure[.]example  (domain)
  VirusTotal   1/90 engines flag as malicious
  URLhaus      hosts 1 malware URL(s), 1 online, tags: js, phishing
  RDAP/WHOIS   registrar Fictional Registrar Ltd, registered 5 days ago (newly registered!)

[PRIVATE] 10[.]20[.]4[.]17  (ipv4)
  internal / non-routable address, external lookups skipped

[CLEAN 0] 1[.]1[.]1[.]1  (ipv4)
  VirusTotal   0/93 engines flag as malicious
  AbuseIPDB    confidence 0%, 12 reports in 90d
  URLhaus      host not listed
  RDAP/WHOIS   APNIC and Cloudflare DNS Resolver project, AU

...
10 IOC(s) checked: 5 malicious, 1 suspicious, 2 clean, 1 unknown, 1 private
```

The full output has 10 indicators: **5 malicious, 1 suspicious, 2 clean, 1 unknown, 1 private**.

Why it matters. Each verdict reflects a habit analysts need. One VirusTotal hit out of 90 is weak on its own, but with a 5-day-old domain and a URLhaus listing the evidence adds up. A public DNS resolver can appear in an incident without being malicious. Unknown doesn't mean safe.

What to say

- “It extracts every indicator from a free-text report, even defanged ones.”
- “Internal IPs, like 10.20.4.17 here, are never sent to external services.”
- “1.1.1.1 is clean: the malware used a public DNS resolver to test its connection. Blocking it would break DNS for the company.”
- “Unknown doesn't mean safe: a brand-new file would go to a sandbox.”

STEP 2 OF 5 ## Detection and automated response ~1 min

YOU RUN

```
$ mini-soar demo
```

WHAT HAPPENS INSIDE



1. **The SIEM reads the logs.** Each source has its own field names and timestamp format (ISO dates, Apache-style dates, Unix timestamps). The SIEM converts them into one common format and uses an asset list to turn IP addresses into computer and user names.
2. **Rules find the attack.** 8 rules written in a Sigma-like format each look for one technique, for example a script running from the Downloads folder, or connections at regular intervals.
3. **Alerts are enriched.** The SIEM sends each alert's indicators to ioc-enricher. An alert with a MALICIOUS indicator goes up one severity level: 5 of the 8 alerts are escalated.
4. **The SOAR picks a playbook per alert. PB-01** for the phishing email, **PB-02** when an indicator is confirmed malicious, **PB-03** for everything else. It blocks the confirmed indicators, opens one case for the infected computer, notifies the team and writes a note into each SIEM alert. Three disruptive actions are left for a human.

WHAT YOU SEE

```
[1/3] mini-SIEM: ingest, detect and enrich the Paper Lantern logs
[2/3] mini-SOAR: run the playbooks
  #2   MS-002  -> PB-02 Contain confirmed malware or C2
  #4   MS-004  -> PB-02 Contain confirmed malware or C2
  #6   MS-006  -> PB-02 Contain confirmed malware or C2
  #1   MS-001  -> PB-01 Phishing response
  #3   MS-003  -> PB-03 Prepare analyst review
  #5   MS-005  -> PB-02 Contain confirmed malware or C2
  #8   MS-008  -> PB-03 Prepare analyst review
  #7   MS-007  -> PB-03 Prepare analyst review

8 alert(s) handled: 28 done, 3 pending, 7 skipped

[3/3] waiting for an analyst:
#2    isolate_host  FIN-WS-07
#18   purge_email   <7f3a@invoice-portal-secure.example>
#31   block_ioc     185.81.157.24
```

Why it matters. The SOAR acts only on what is *confirmed*. Blocking on suspicion breaks legitimate services, and isolating a computer stops someone from working, so those decisions wait for a person.

What to say

- “The SIEM raised 8 alerts, and 5 were escalated because threat intel confirmed an indicator.”
- “The SOAR blocked 4 confirmed indicators automatically.”
- “It left 3 decisions for a human: isolate the computer, delete the phishing email, and block an IP that was only rated suspicious.”

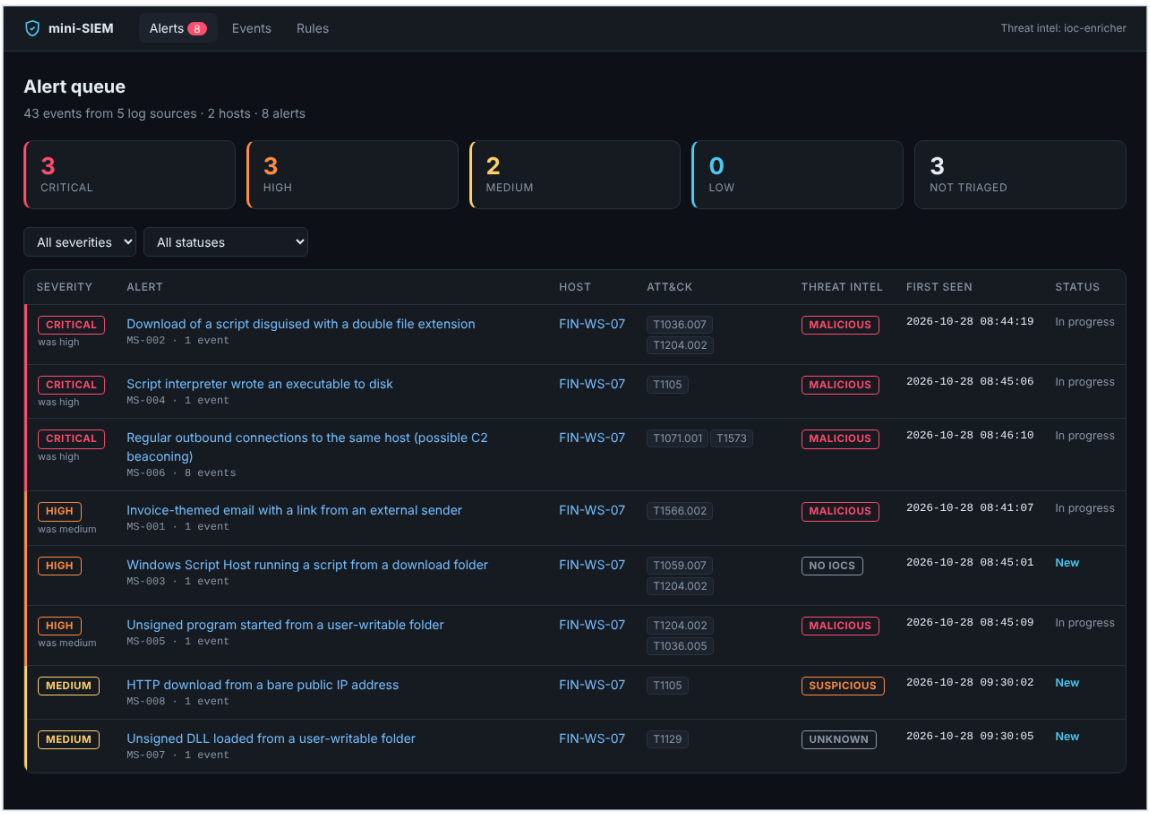
STEP 3 OF 5 The analyst's view in the dashboard ~3–4 min

YOU RUN (THE SCRIPT DOES IT IN THE BACKGROUND)

```
$ mini-siem --db mini-soar\soar-output\mini_siem.db serve
```

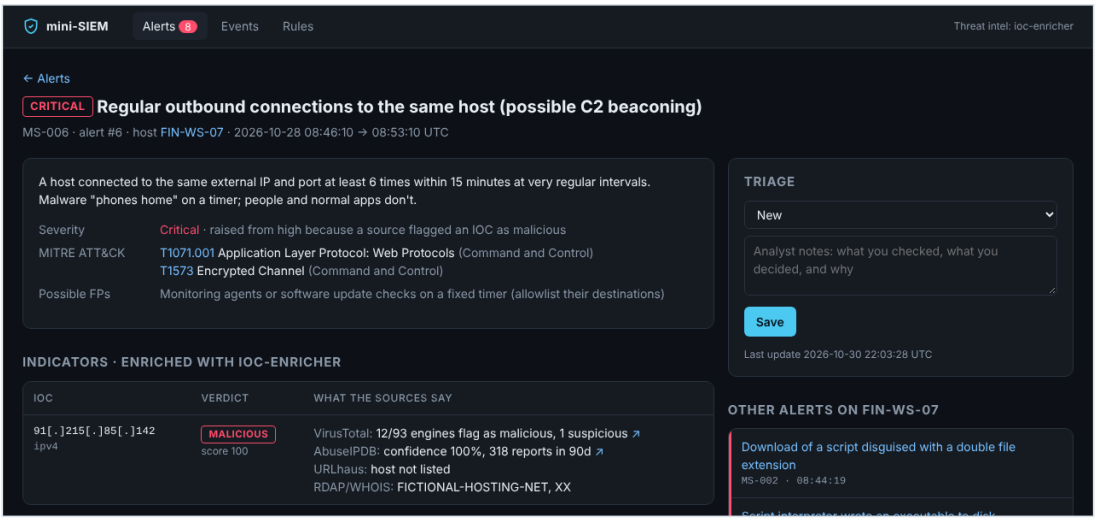
The dashboard is a small website running on your own computer, at `http://127.0.0.1:5000`. Only your computer can reach it. The script opens four pages in turn:

PAGE 1: THE ALERT QUEUE



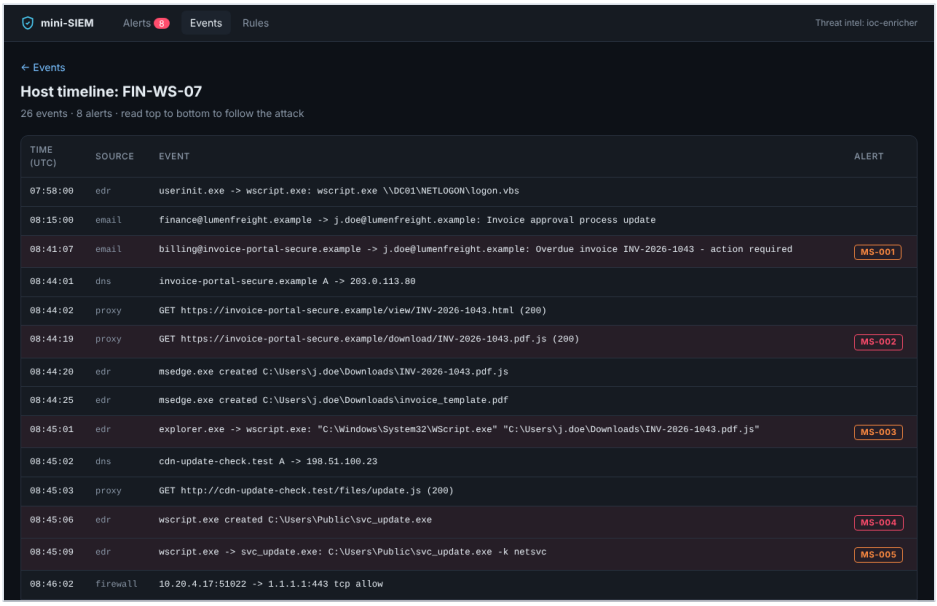
1. **Severity and “was high”.** The coloured badge is the current severity; “was high” means threat-intel enrichment raised it.
2. **Threat intel column.** The worst verdict among the alert's indicators. “No IOCs” means the alert has no external indicator (for example a process running on the computer).
3. **Status. In progress**: the SOAR has acted. **New**: uncertain alerts left for an analyst.

PAGE 2: THE C2 BEACONING ALERT



This rule doesn't look at one event. It needs **at least 6 connections to the same IP within 15 minutes, at regular intervals**, because malware “phones home” on a timer while people and normal apps don't. The page shows what each threat-intelligence source says about the IP, the MITRE ATT&CK techniques, the 8 firewall logs behind the alert, and the triage box where the SOAR's note appears.

PAGE 3: THE HOST TIMELINE



Every event of the infected computer in time order, with each alert next to the event that caused it: you can read the whole attack from top to bottom. **Page 4** lists the 8 rules with their ATT&CK mapping.

Why it matters. A rule that catches the attack but also fires on normal traffic is useless in a SOC. The look-alike traffic from the second workstation (Teams, a CDN script, a logon script) raises nothing, and the tests check it.

What to say

- “Confirmed alerts are In progress: the SOAR acted. Uncertain ones stay New for an analyst.”
- “The beaconing rule checks regularity, not just volume, so Teams traffic doesn't trigger it.”
- “The timeline tells the whole attack, from the email to the second stage.”

STEP 4 OF 5 ## Human in the loop: approve and reject ~1–2 min

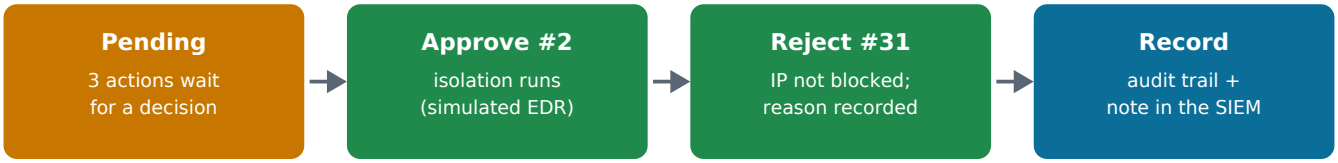
YOU RUN

```
$ mini-soar pending
```

```
$ mini-soar approve 2 --by mateo
```

```
$ mini-soar reject 31 --by mateo --reason "shared hosting, monitor instead"
```

WHAT HAPPENS INSIDE



1. **Approving.** The SOAR checks the action is still needed (the computer isn't already isolated), runs it, and writes the request to `edr_actions.jsonl`. It's **simulated**: there is no real EDR connected.
2. **Rejecting.** The IP stays unblocked, and your reason is kept.
3. **Recording.** Both decisions go into the audit trail with your name and the time, into the SIEM alert's notes, and into the incident case's list of open approvals.
4. **The case file.** `soar-output\cases\CASE-0001.md` groups all 8 alerts of the incident, instead of 8 separate tickets.

WHAT YOU SEE

```
#2 approved: approved by mateo: SIMULATED: isolation request written to edr_actions.jsonl (no real EDR connected)
#31 rejected by mateo: shared hosting, monitor instead
```

Why it matters. Automation should prepare the analyst's work, not override it. The SOAR never overwrites analyst notes, never reopens a closed alert, and never runs a disruptive action without a named person approving it.

What to say

- “Isolating a computer stops someone working, so a human decides. It's simulated here.”
- “I rejected the IP block: it was only suspicious, and shared hosting can serve legitimate sites.”
- “Everything is in an audit trail: what, on which target, by whom and when.”

STEP 5 OF 5 ## How it is tested ~30 s

YOU RUN

```
$ pytest -q
```

WHAT HAPPENS INSIDE

pytest runs the 115 tests of the mini-SIEM, all offline. The most important one replays the 43 Paper Lantern events and checks that **each event triggers exactly the rules it should**: the attack events trigger their rule, and the benign look-alike events trigger nothing. Across the three projects there are 271 tests, and GitHub Actions runs them, plus this demo on Windows and Linux, at every push.

WHAT YOU SEE

```
.................................................................... [ 62%]
.........................................                            [100%]
115 passed in 2.21s
```

What to say

- “Each of the 43 log events is checked against the rules it should trigger, including the ones that must stay silent.”
- “The tests run automatically on GitHub at every push.”

After the demo: what was created

Everything the demo produces is in `mini-soar\soar-output\`. It is rebuilt from scratch every time you run the demo, so you can run it as often as you like.

File	What it contains
`mini_siem.db`	The SIEM database: events, alerts, enrichment results
`blocklists\ips.txt, domains.txt, urls.txt, hashes.txt`	Indicators blocked by the SOAR, ready for a firewall or proxy
`cases\CASE-0001.md`	The incident case: all 8 alerts, indicators, events, actions, open approvals
`notifications.log`	The messages that would go to Discord or Slack
`edr_actions.jsonl`	The simulated isolation you approved
`..\mini_soar.db`	The SOAR's audit trail (mini-soar history)

```
# mini-soar ips blocklist: one indicator per line, comments after '#'
91.215.85.142  # MALICIOUS, MS-006 alert #6, 2026-11-03T21:24:36Z
```

Example: `blocklists\ips.txt`, with why and when each entry was added.

Glossary

SOC	Security Operations Centre: the team that monitors an organisation's systems and responds to incidents.
SIEM	Security Information and Event Management: collects and correlates logs, raises alerts.
SOAR	Security Orchestration, Automation and Response: runs playbooks that enrich, contain and document alerts.
EDR	Endpoint Detection and Response: security agent on each computer; records processes and files, can isolate a machine.
IOC	Indicator of compromise: an IP, domain, URL or file hash linked to malicious activity.
Defanging	Writing an IOC so it isn't clickable, e.g. hxxp://evil[.]com, so nobody opens it by accident.
Enrichment	Adding context to an alert, such as threat-intelligence verdicts, owners, and asset names.
Threat intelligence	Shared knowledge about malicious infrastructure: VirusTotal, AbuseIPDB, URLhaus...
MITRE ATT&CK	Public catalogue of attacker tactics and techniques; each rule is mapped to one, e.g. T1071.001.
Sigma	A common format for writing detection rules; the mini-SIEM uses a simplified version.
C2 / beaconing	Command-and-control: malware contacting its server, often at regular intervals (“beacons”).
Playbook	A predefined sequence of response steps for a type of alert.
Human in the loop	Automated steps that wait for a person's approval before running.
False positive	An alert on activity that is actually legitimate.
Audit trail	A record of who did what, when: needed to review decisions and for compliance.
Phishing	A deceptive message that tricks someone into clicking a link, opening a file or giving credentials.

If something goes wrong

Problem	Fix
PowerShell refuses to run the script	Run once: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
“Missing: ...” or an import error	Run `.\demo.ps1 -Setup` to reinstall the projects
The browser doesn't open	Open the address the script prints, e.g. http://127.0.0.1:5000/alerts
Port 5000 is busy	Nothing to do: the script picks the next free port and tells you which
“file in use” on soar-output	A dashboard from an earlier run is still open: close that window and retry
Anything else during an interview	Stay calm and switch to the screenshots in the READMEs: explaining from them works too